Cursores en PL/SQL

Recorrido de conjuntos de filas desde PL/SQL

Bases de Datos · Ingeniería de Sistemas · Oracle Database 19c

Cursores

Bases de Datos · Ingeniería de Sistemas · PL/SQL

Dónde estamos

- Ya sabemos escribir bloques, declarar variables y ejecutar sentencias SQL desde PL/SQL. Falta responder una pregunta práctica: qué hacer cuando la consulta devuelve muchas filas.
- El cursor es la respuesta, y no es una sola construcción sino cinco, cada una para un problema distinto.

Recorrido

- El cursor implícito: el que usted ya viene usando sin saberlo
- El cursor explícito y su ciclo de vida completo
- Ciclo FOR, cursores parametrizados y actualización posicionada
- Cursores de referencia y procesamiento masivo

El problema que resuelve el cursor

Bases de Datos · Ingeniería de Sistemas · PL/SQL

Dos mundos que no encajan

- SQL trabaja con conjuntos: una consulta devuelve N filas de una sola vez.
- PL/SQL trabaja con variables escalares: una variable guarda un solo valor.
- El cursor es el área de trabajo donde Oracle deja el resultado y desde la cual PL/SQL lo consume fila por fila.

La consecuencia

- Toda sentencia SQL ejecutada desde PL/SQL abre un cursor, lo declare usted o no.
- La decisión nunca es si usar cursor: es cuál de los cinco.

Regla de partida: si el problema se resuelve con una sola sentencia SQL, no use cursor. El cursor entra cuando hay lógica por fila.

Cursor implícito

Oracle lo abre y lo cierra. El programador solo consulta sus atributos.

```
BEGIN
    UPDATE producto
    SET    precio_unitario = precio_unitario * 1.05
    WHERE  id_categoria = 1;

    DBMS_OUTPUT.PUT_LINE('Filas afectadas: ' || SQL%ROWCOUNT);

    IF SQL%NOTFOUND THEN
        DBMS_OUTPUT.PUT_LINE('Ninguna fila cumplia la condicion');
    END IF;
END;
```

SELECT ... INTO también abre un cursor implícito: es el que levanta NO_DATA_FOUND y TOO_MANY_ROWS.

Atributos del cursor implícito

Atributo	Tipo	Qué devuelve
SQL%FOUND	BOOLEAN	Verdadero si la última sentencia afectó al menos una fila
SQL%NOTFOUND	BOOLEAN	Verdadero si no afectó ninguna fila
SQL%ROWCOUNT	NUMBER	Cantidad de filas afectadas por la última sentencia
SQL%ISOPEN	BOOLEAN	Siempre falso: Oracle cierra el cursor implícito de inmediato

Los atributos se refieren SIEMPRE a la última sentencia ejecutada. Consultarlos después de otra sentencia devuelve el valor equivocado.

Cursor explícito: cuatro fases

Fase	Sentencia	Qué ocurre
Declaración	CURSOR c IS SELECT ...	Se define la consulta; todavía no se ejecuta
Apertura	OPEN c;	Se ejecuta la consulta y el puntero queda antes de la primera fila
Recuperación	FETCH c INTO ...;	Trae una fila y avanza el puntero
Cierre	CLOSE c;	Libera el área de trabajo y los recursos asociados

No cerrar el cursor agota el parámetro OPEN_CURSORS de la sesión y termina en ORA-01000.

Cursor explícito: ejemplo completo


```
DECLARE
    CURSOR c_bajo_minimo IS
        SELECT codigo, nombre, stock_total, stock_minimo
        FROM   v_stock_consolidado
        WHERE  stock_total < stock_minimo;

    r_prod c_bajo_minimo%ROWTYPE;
BEGIN
    OPEN c_bajo_minimo;
    LOOP
        FETCH c_bajo_minimo INTO r_prod;
        EXIT WHEN c_bajo_minimo%NOTFOUND;
        DBMS_OUTPUT.PUT_LINE(r_prod.codigo || ' faltan ' ||
                             (r_prod.stock_minimo - r_prod.stock_total));
    END LOOP;
    CLOSE c_bajo_minimo;
END;
```

Atributos del cursor explícito

Atributo	Antes de OPEN	Después de OPEN	Tras el último FETCH
c%ISOPEN	FALSE	TRUE	TRUE hasta el CLOSE
c%FOUND	ORA-01001	NULL	FALSE
c%NOTFOUND	ORA-01001	NULL	TRUE
c%ROWCOUNT	ORA-01001	0	Total de filas recuperadas

El orden importa: EXIT WHEN c%NOTFOUND va inmediatamente después del FETCH. Invertirlo procesa la última fila dos veces.

Ciclo FOR sobre cursor

El mismo recorrido, sin las cuatro fases explícitas.

```
DECLARE
    CURSOR c_prod IS
        SELECT codigo, nombre FROM producto WHERE activo = 'S';
BEGIN
    FOR r IN c_prod LOOP
        DBMS_OUTPUT.PUT_LINE(r.codigo || ' - ' || r.nombre);
    END LOOP;
END;
```

- Oracle declara la variable de registro, abre, recupera, evalúa la salida y cierra.
- Cierra el cursor incluso si el bloque termina por excepción.

Explícito o FOR: criterio de elección

Use el ciclo FOR

- Recorrido completo del resultado
- Sin lógica de salida anticipada
- Código más corto, sin fugas de cursor
- Es el caso más frecuente en producción

Use el cursor explícito

- Necesita %ROWCOUNT durante el recorrido
- Debe salir bajo una condición propia
- Recupera desde varios puntos del código
- Requiere control fino de apertura y cierre

En el curso se exige el explícito aunque el FOR baste: hace visible el ciclo de vida que el FOR oculta.

Cursores parametrizados

Una definición, muchas ejecuciones con valores distintos.

```
DECLARE
    CURSOR c_pedidos (p_cliente NUMBER, p_desde DATE) IS
        SELECT id_pedido, fecha_pedido, total
        FROM   pedido
        WHERE  id_cliente = p_cliente
        AND    fecha_pedido >= p_desde
        AND    estado = 'CONFIRMADO';
BEGIN
    FOR r IN c_pedidos(501, DATE '2026-01-01') LOOP
        DBMS_OUTPUT.PUT_LINE(r.id_pedido || ' : ' || r.total);
    END LOOP;
END;
```

Los parámetros son de solo lectura y solo de entrada. No sustituyen nombres de tabla ni de columna.

FOR UPDATE y WHERE CURRENT OF

Bases de Datos · Ingeniería de Sistemas · PL/SQL

Leer con intención de modificar.

```
DECLARE
    CURSOR c_prod IS
        SELECT id_producto, precio_unitario
        FROM   producto
        WHERE  id_categoria = 1
        FOR UPDATE OF precio_unitario;
BEGIN
    FOR r IN c_prod LOOP
        UPDATE producto
        SET    precio_unitario = r.precio_unitario * 1.07
        WHERE  CURRENT OF c_prod;
    END LOOP;
    COMMIT;
END;
```

Qué bloquea y por cuánto tiempo

Alcance del bloqueo

- OPEN con FOR UPDATE bloquea todas las filas del resultado, no solo la que se está procesando.
- El bloqueo dura hasta el COMMIT o el ROLLBACK, no hasta el CLOSE.
- WHERE CURRENT OF actualiza la fila posicionada por el cursor, sin reevaluar la clave primaria.

Variantes

- FOR UPDATE NOWAIT falla de inmediato si otra sesión tiene las filas.
- FOR UPDATE SKIP LOCKED omite las filas bloqueadas: es el patrón de cola de trabajo.

Un cursor FOR UPDATE sobre un resultado amplio puede bloquear media tabla durante toda la transacción.

Cursores de referencia

El cursor como valor que se puede devolver.

```
CREATE OR REPLACE PROCEDURE sp_listar_movimientos (
    p_id_producto IN  NUMBER,
    p_cursor      OUT SYS_REFCURSOR
) IS
BEGIN
    OPEN p_cursor FOR
        SELECT fecha_mov, tipo, cantidad
        FROM   movimiento_inventario
        WHERE  id_producto = p_id_producto
        ORDER  BY fecha_mov;
END;
```

Quien abre el cursor no es quien lo recorre. El llamador hace FETCH y es responsable del CLOSE.

Cursor estático frente a cursor de referencia

Bases de Datos · Ingeniería de Sistemas · PL/SQL

Aspecto	Cursor estático	SYS_REFCURSOR
Consulta	Fija en tiempo de compilación	Puede construirse en ejecución
Alcance	Local al bloque que lo declara	Se pasa como parámetro entre programas
Validación	El compilador verifica la consulta	Se valida al abrirlo
Uso típico	Lógica interna de un programa	Devolver resultados a la aplicación

El cursor de referencia es el mecanismo estándar para entregar un resultado a Java, .NET o Python.

BULK COLLECT

Bases de Datos · Ingeniería de Sistemas · PL/SQL

Traer muchas filas en una sola operación.

```
DECLARE
    TYPE t_ids IS TABLE OF producto.id_producto%TYPE;
    v_ids t_ids;
BEGIN
    SELECT id_producto BULK COLLECT INTO v_ids
    FROM   producto
    WHERE  id_categoria = 3;

    DBMS_OUTPUT.PUT_LINE('Recuperados: ' || v_ids.COUNT);
END;
```

Sin la cláusula LIMIT, BULK COLLECT carga TODO el resultado en memoria de sesión. En tablas grandes se recupera por lotes.

FORALL

Enviar muchas sentencias DML en una sola pasada.

```
FORALL i IN 1 .. v_ids.COUNT
    UPDATE producto
    SET    stock_minimo = CEIL(stock_minimo * 1.15)
    WHERE  id_producto = v_ids(i);

DBMS_OUTPUT.PUT_LINE('Actualizadas: ' || SQL%ROWCOUNT);
```

- FORALL no es un ciclo: es una sola instrucción enviada al motor SQL con todo el conjunto.
- No admite lógica en su cuerpo, únicamente una sentencia DML.
- SAVE EXCEPTIONS permite continuar ante filas con error y revisarlas al final.

Por qué el procesamiento masivo es más rápido

El costo oculto

- PL/SQL y SQL son dos motores distintos dentro de la misma instancia.
- Cada FETCH o cada DML individual implica un cambio de contexto entre ambos.
- Un ciclo de cien mil filas produce cien mil cambios de contexto.
- BULK COLLECT y FORALL reducen esa cifra a una o a unas pocas.

Cuándo importa

- La ganancia aparece con el volumen: por debajo de unos cientos de filas es despreciable.
- El cambio de contexto no es el costo de la consulta: es el costo de cruzar la frontera entre los dos motores.

Errores frecuentes

Error	Consecuencia
No cerrar el cursor explícito	ORA-01000: se agota el máximo de cursores abiertos
EXIT WHEN antes del FETCH	La última fila se procesa dos veces
Usar cursor donde bastaba una sentencia SQL	Orden de magnitud de diferencia en tiempo
FOR UPDATE sobre un resultado amplio	Bloqueo prolongado sobre gran parte de la tabla
BULK COLLECT sin LIMIT en tablas grandes	Consumo de memoria y ORA-04030
Modificar dentro del ciclo la tabla recorrida	Resultados inconsistentes o tabla mutante

Para retener

- Todo SQL ejecutado desde PL/SQL abre un cursor; la decisión es cuál.
- El ciclo FOR resuelve el caso común; el explícito da control y hace visible el ciclo de vida.
- FOR UPDATE bloquea al abrir y libera al confirmar, no al cerrar.
- El cursor de referencia sirve para entregar resultados fuera de la base de datos.
- BULK COLLECT y FORALL atacan el cambio de contexto, no la consulta.

En el taller: los cinco tipos aparecen en la Parte 3, y el criterio de elección es parte de la calificación.